How SSH works

What happens when you type `ssh ws`, and what your two keys are for.

The one-minute version

- SSH opens a terminal on a faraway computer, and keeps everything you type and see **private** on the way.
- You make a **key pair**: a **private key** that never leaves your laptop, and a **public key** you give to the server.
- To log in, your laptop **proves it holds the private key** without ever sending it. No password travels anywhere.
- The server also proves who it is, so you know you are talking to *your* server and not an impostor.

---

1 Why SSH exists

Your course server is a computer in a data centre in Iowa. It has no keyboard or screen you can reach. **SSH** (Secure Shell) lets your laptop open a terminal on that computer across the internet: what you type goes to the server, and its output comes back to your screen.

The internet is not private. Your data passes through many machines that belong to other people. The old tools for remote terminals, such as `telnet`, sent everything, passwords included, as plain text. SSH fixes two problems:

- **Privacy.** The traffic is encrypted. Someone in the middle sees only noise.
- **Identity.** Each side checks who the other is. You check that this is really your server, and the server checks that you are really you.

---

2 Two keys, one idea

A **key pair** is created together, in one command (`ssh-keygen`). It gives you two files that are mathematically linked:

Private key: `id_ed25519`

- Stays on your laptop, always.
- Never send it, paste it, email it or commit it.
- It lets you **prove who you are**.

Public key: `id_ed25519.pub`

- One line of text. Safe to share with anyone.
- You paste it into servers (and GitHub) that should let you in.
- It lets others **check your proof**, but not make it.

An analogy: the specimen signature at a bank

When you open a bank account, the bank keeps a **specimen of your signature**. Later, anyone can compare a signature on a cheque with the specimen, but only you can *produce* a matching signature. The specimen is your **public key**. Your hand, and its ability to sign, is your **private key**. The difference from a real signature: a digital signature cannot be forged, and the specimen reveals nothing that would help to copy it.

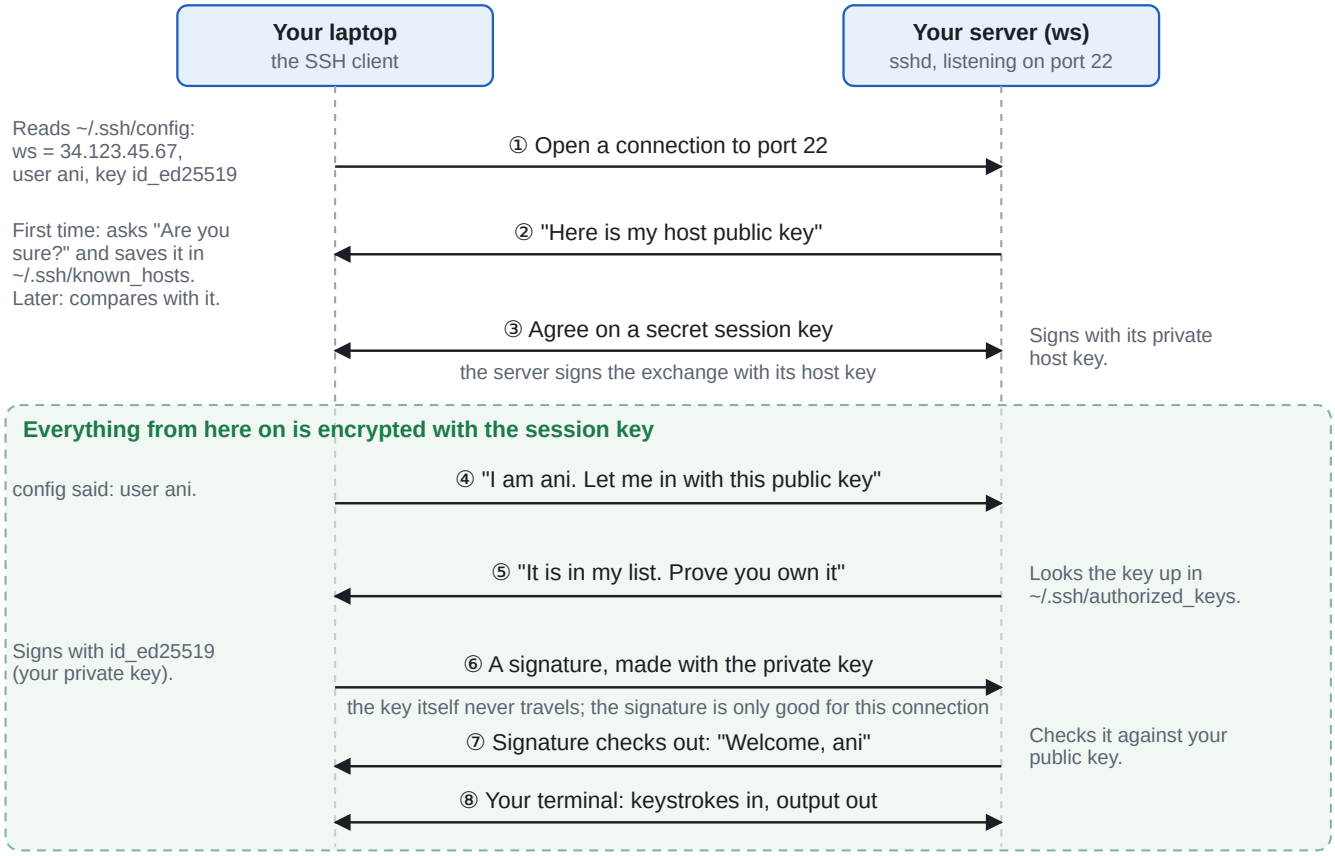
What really happens is a **digital signature**. Something signed with the private key can be verified with the public key. The public key cannot create signatures, and the private key cannot be worked out from it. The kind of key we use, **Ed25519**, is a modern, short and fast signature scheme.

Why this beats a password

	Password	Key pair
What the server stores	Something that can check your secret. If it leaks, the secret is exposed.	Only your **public** key. Useless to a thief.
What crosses the network	The secret itself (inside the encrypted channel).	Only a signature, good for this one connection.
Guessing	People pick guessable passwords, and bots try millions of them.	A 256-bit key cannot be guessed.
Convenience	You type it every time.	Nothing to type.

---

3 What happens when you type `ssh ws`



One SSH login, step by step. Dashed vertical lines are time moving down; each arrow is a message.

1. **Look up the nickname.** `ws` is not a real address. Your laptop reads `~/.ssh/config` and finds the real IP, the user name and the key to use. (Typing `ssh ani@34.123.45.67` does the same without the nickname.) Then it connects to **port 22**, the standard SSH port. If the server is stopped, or a firewall blocks the port, you wait and get *Connection timed out*.

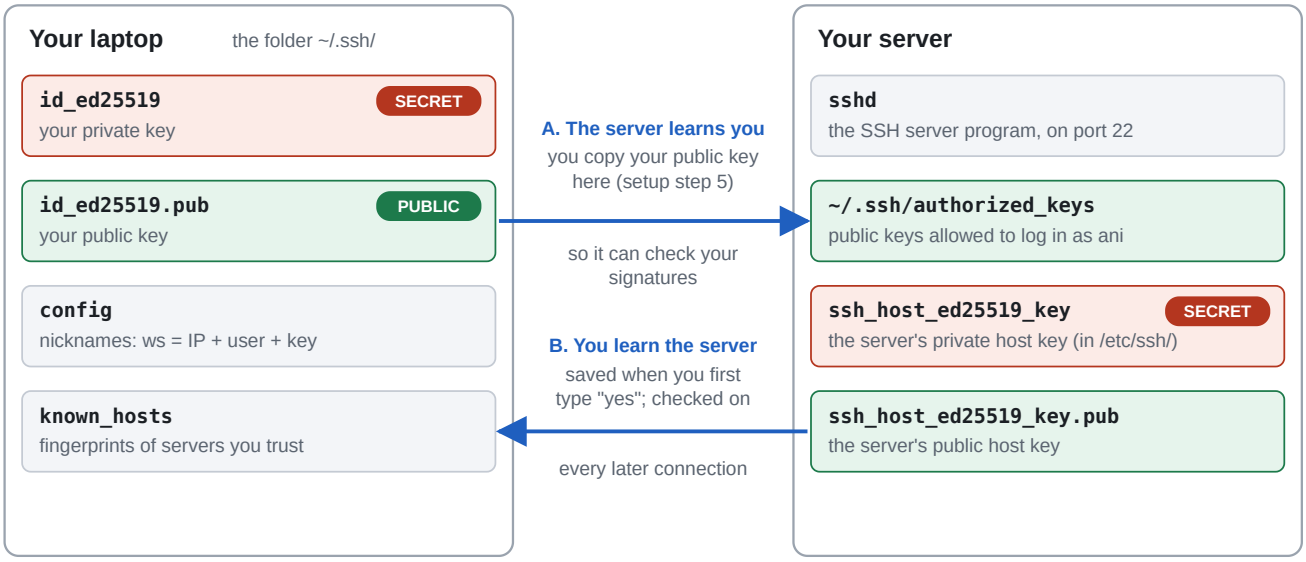
2. **The server shows who it is.** Every server has its own **host key pair**, made when it was first set up. It sends the public half. The first time you connect, your laptop cannot know whether this is the right server, so it asks *"Are you sure you want to continue connecting?"* and, when you say `yes`, stores the key's **fingerprint** (a short hash of it) in `~/.ssh/known_hosts`. From then on it checks automatically.

3. **Agree on a secret, then encrypt.** The two sides run a **key exchange** (Diffie–Hellman): each sends a little public information, and from it both compute the *same* secret **session key**, which is never sent over the network. The server **signs** the exchange with its host key, so you know the one you agreed the secret with is the real server. From now on, everything is encrypted with the session key using a fast **symmetric cipher**. (Public-key maths is slow, so it is used only to set up and to prove identity; the session key does the bulk work.)
4. **You say who you are.** Your laptop sends the user name and offers a public key. The server looks in that user's `~/.ssh/authorized_keys` file, the list of public keys allowed to log in.
5. **You prove it.** If the key is listed, the server asks you to prove you own the private half. Your laptop **signs** a value tied to this very connection and sends the signature. The private key itself never leaves your laptop, and a recorded signature is useless later because it only fits this one connection.
6. **You are in.** The server checks the signature against your public key and starts a shell for you. The prompt changes to `ani@ws:~$`: everything you type now runs on the server.

---

4 Who trusts whom: two directions

SSH checks identity **both ways**, and each direction uses its own pair of keys and its own file:



Arrow A makes the server trust you. Arrow B makes you trust the server. Private keys (red) never move.

On Windows the same files live in `C:\Users\<you>\.ssh\`. The OpenSSH client that ships with Windows 10 and 11 works the same way as on macOS and Linux.

5 How this maps to your setup

Setup step	What it does in SSH terms
Step 2: `ssh-keygen`	Makes your key pair. The part after `-C` is only a label; it ends up at the end of the public key line.
Step 5: paste the public key into the server form	Google Cloud's agent on the server creates the user `ani` and adds your public key to `~/.ssh/authorized_keys`. This is arrow A. The user name is the server's own account; it has nothing to do with your Google account.
Step 5: the static IP	Keeps the server's address the same, so the `HostName` in your config stays valid after you stop and start the server.
Step 6: the `config` file	Gives the nickname `ws`.
The first `ssh ws`: type `yes`	Saves the server's host key in `known_hosts`. This is arrow B.
The default firewall rule	Port 22 is the one port open to the internet. If you later want a web page (for example Open WebUI), either open that port in the firewall or reach it through an SSH tunnel (see section 8).

6 Keeping your keys safe

- **The private key (the file without `.pub`) never leaves your laptop.** Do not email it, paste it into a chat, give it to an AI assistant, or commit it to git. If it ever leaves, treat it as stolen: remove its public key from the server and make a new pair.
- **The public key line is safe to paste anywhere.** That is its job.
- **A passphrase** is an optional password that locks the private key file on your disk, so a stolen file is useless without it. The setup guide leaves it empty to keep things simple. For servers that matter, use one, together with `ssh-agent`, which remembers it for you while you work.
- **Lock your laptop and encrypt its disk.** With an empty passphrase, whoever can read your files can log in as you.
- **Lost or sold a laptop?** Remove its public key from `authorized_keys` on the server. A new laptop means a new key pair.
- OpenSSH refuses a private key that other users can read. If you see *UNPROTECTED PRIVATE KEY FILE*, run `chmod 600 ~/.ssh/id_ed25519`.

7 When it goes wrong

You see	What it means and what to do
`Could not resolve hostname wc`	SSH took the word you typed as a real hostname because there is no `Host` entry with that name. Check the spelling (`ws`, not `wc`) and that the `config` file exists and is saved without a `.txt` ending.
`Connection timed out`	Nothing answered. The server is stopped, the IP in `config` is wrong, or a firewall blocks port 22. Start the server and compare the IP.
`Connection refused`	The machine answered but nothing listens on port 22: usually the server is still booting. Wait a minute.
`Permission denied (publickey)`	The server did not accept your key: the wrong user name, the wrong key pasted in step 5, or SSH offered a different key. Run `ssh -v ws` and look at which key it offers.

You see	What it means and what to do
`Too many authentication failures`	SSH offered many keys one by one and the server gave up before reaching the right one. In your `Host` block add `IdentityFile ~/.ssh/id_ed25519` and `IdentitiesOnly yes`.
`REMOTE HOST IDENTIFICATION HAS CHANGED`	The server at this address shows a different host key than the one you saved. After you delete and recreate a server, that is expected. If you did not, be suspicious. When sure, forget the old key with `ssh-keygen -R 34.123.45.67`.

8 What else SSH can do

- **Run one command** without opening a shell: `ssh ws 'df -h'`
- **Copy files:** `scp notes.txt ws:~/` (to the server) and `scp ws:~/results.csv .` (back to you). `rsync` and `sftp` use the same connection.
- **Tunnel a port.** `ssh -L 3000:localhost:3000 ws` makes `http://localhost:3000` in your laptop's browser reach port 3000 *on the server*, through the encrypted connection. Nothing has to be opened in the firewall.
- **Other tools reuse your setup.** VS Code's Remote–SSH, and `git` over SSH, read the same `config` and keys. GitHub also accepts public keys: the same idea, for `git push`.

9 Try it yourself (5 minutes)

1. On your laptop, look at your nicknames: `cat ~/.ssh/config` (Windows: `type $env:USERPROFILE\.ssh\config`). Find the `Host ws` block.
2. Show your public key's fingerprint: `ssh-keygen -lf ~/.ssh/id_ed25519.pub`
3. Watch the login happen: `ssh -v ws`. Find the lines *Server host key*, *Offering public key* and *Authenticated to … using "publickey"*. Which numbered step in section 3 is each one?
4. On the server, find your public key in the list: `cat ~/.ssh/authorized_keys`
5. **Think:** why is it safe to paste your *public* key into a web form, but never your *private* key?

10 Words to know

SSH	Secure Shell: an encrypted way to use a remote computer's terminal.
Key pair	A private key and a public key, made together.
Private key	The secret half. Lets you create signatures. Never shared.
Public key	The shareable half. Lets others check your signatures.
Signature	Proof, made with a private key, that can be checked with the public key.
Host key	The server's own key pair: how a server proves its identity.
Fingerprint	A short hash of a key, so people can compare keys at a glance.
Session key	The temporary secret both sides compute; it encrypts one connection.
`authorized_keys`	On the server: public keys allowed to log in as that user.
`known_hosts`	On your laptop: host keys of servers you have trusted.
Passphrase	An optional password that locks the private key file.
Port	A numbered door on a computer. SSH uses 22.